



ThunderKittens：简单、高速、可爱的 AI 内核

Benjamin F. Spector, Simran Arora, Aaryan Singhal, Daniel Y. Fu, Christopher Ré

斯坦福大学

{bfs, simarora, aaryan04, danfu, chrismre}@stanford.edu

2024年10月29日

摘要

将 AI 架构映射到 GPU 硬件的挑战正成为 AI 进展的关键瓶颈。尽管投入了大量努力，手写的定制内核仍未能达到其理论性能上限，即使对于诸如线性注意力等已确立的操作也是如此。GPU 硬件多样的能力似乎意味着我们需要各种技术才能实现高性能。然而，我们的工作探讨了少数几个关键抽象能否极大地简化这一过程。我们提出了 ThunderKittens (TK)，一个用于编写高性能 AI 内核的框架，同时保持易用性和可维护性。我们的抽象对应于 GPU 层次结构的三个级别：(1) 在线程束级别 (warp-level)，我们提供 16×16 矩阵分块 (tile) 作为基本数据结构，并提供类似 PyTorch 的分块并行计算操作；(2) 在线程块级别 (thread-block level)，我们提供一个模板用于跨并行线程束重叠异步操作；(3) 在网格级别 (grid-level)，我们提供支持来帮助隐藏线程块的启动、终止以及内存访问的开销。我们通过在多种 AI 操作上提供与现有内核性能相当或更优的内核来展示 TK 的价值。在 GEMM 和注意力推理性能方面，我们达到了与 CuBLAS 和 FlashAttention-3 相当的水平；在注意力反向传播上，我们比最强基线高出 10-40%，在状态空间模型上高出 8 倍，在线性注意力上高出 14 倍。

1 引言

AI 的发展受限于如何将 AI 架构高效映射到 GPU 硬件上这一问题。ML 架构正经历“寒武纪爆发”式的激增【21, 25】；然而，即使经过大量努力，这些架构的实际性能仍远低于其理论潜力。值得注意的是，即使对于工业界广泛使用的 Softmax 注意力，kernel 支持也一直不佳。FlashAttention-2【12】在移植到 H100 GPU 时性能下降了 47%，而从 H100 发布到开发出 FlashAttention-3【37】则花费了两年多的时间。

我们从多种支持 AI 内核开发的方法中汲取灵感。理想情况下，我们期望一个框架能支持**广泛**的算子并达到**高性能**，同时又**易于使用、学习和维护**。高性能的 C++ 嵌入式库（如 NVIDIA 的 CUTLASS/CuTe【29】）包含大量嵌套模板；而编译器方法（如 Triton【39】）为用户提供了更简单的接口，但优化手段较少。我们的问题是：如果选择一小套精心设计的抽象，能在多大程度上兼顾广度和速度？

当前加速计算的主要发展方向是专用的矩阵乘法单元。在 NVIDIA A100 和 H100 GPU 上，BF16 张量核心 (Tensor Core) 的 FLOPS 性能是通用 BF16/FP32 单元的 16 倍。因此，任何高性能框架都必须尽可能优先保持张量核心的高利用率。然而，所有 kernel 都还有其他操作（例如内存加载或注意力中的 softmax），因此关键在于将非张量核心操作的开销降到最低。这一观点是我们方法的核心。

为了理解构建一个简单而高性能框架的复杂性和契机，我们考察了一个简化的 GPU 并行性模型，详细内容见第 2.1 节¹。

图1： ThunderKittens 探索少量关键抽象能否支持高效且简单的 AI 内核。受 PyTorch 启发，我们的抽象包括具有管理布局的分块及其上的操作。我们提供了一个通用程序模板，用于协调异步并行工作线程——例如，一些工作线程从 HBM（高带宽内存）加载和存储数据，而其他工作线程在高速内存中执行计算。

1. **线程束级别并行：**现代 GPU 包含成千上万的硬件线程可并行执行。这些线程被组织成小组，即“warp（线程束）”，一个 warp 内的线程同步执行指令。内存布局决定了逻辑数据元素如何映射到物理线程的所属关系。如果多个线程尝试访问同一内存“bank（存储体）”区域，就会在线程间产生昂贵的串行化（称为“存储体冲突”）。
2. **线程块级别并行：**线程束被分组成线程“块”，这些线程块内的线程可以快速共享数据。线程束在物理执行单元上执行指令，在一个块中增加更多 warp（称为占用率）可以让更多指令同时运行，从而减少运行时间。例如，一个 warp 可在张量核心上执行矩阵乘法，而另一个 warp 使用算术逻辑单元（ALU）计算最大值。
3. **网格级别并行：**GPU 上会同时运行多个线程块（组成一个网格），它们通过容量大但速度较慢的全局内存（HBM）进行通信。片上共享的 L2 缓存可以降低存储延迟、提高带宽，前提是线程块重用相同数据。如果线程块重复使用数据，L2 缓存通常可以提供比 HBM 快得多的访问。但是线程块的启动和结束也有开销，称为“流水线气泡（pipeline bubbles）”。

尽管表面上看来需要大量技巧来充分利用上述所有硬件能力，我们的核心技术发现是：对于许多 AI kernel 而言，确实存在少数关键抽象可以简化高性能 kernel 的编写过程。我们的探索促使我们开发了 ThunderKittens (TK)，一个围绕以下三项关键原则构建的 AI 内核框架：

1. **具有管理布局的分块数据结构：**我们的接口借鉴了 PyTorch 和 NumPy [31] 等常用 ML 框架（如图2所示）。在线程束级别，我们使用 16×16 的矩阵分块作为基本数据结构，以最大程度兼容并鼓励使用张量核心。TK 会自动为分块选择最优的内存布局，以尽量减少存储体冲突，同时又保持与专用硬件指令的兼容，从而免除用户工作。我们提供了一组分块上的并行计算原语，这些原语基于 PyTorch 的操作集合（例如，按元素乘法、矩阵乘累加 mma、指数 exp、累积和 cumsum 等分块操作）。
2. **用于异步工作的程序模板：**在线程块级别，TK 提供了一个通用 kernel 模板，用于在线程块内协调跨 warp 的异步执行，它建立在经典的生产者-消费者范式 [16] 之上。开发者只需在该模型中填充少量样板函数，使用我们类 PyTorch 的操作数；模板内部通过内存流水线和同步原语隐藏延迟（如图1所示）。
3. **用于线程块流水的网格调度：**在网格级别，我们展示了 TK 如何帮助开发者减少流水线气泡并提高 L2 缓存命中率。我们的模板支持持久化网格（persistent grid），通过在线程块边界间重叠内存加载来减少块启动的间隙。

【图2embed_image】 PyTorch 实现：

```
# imports
import torch
import torch.nn.functional as F

# compute Q@K.T
att = torch.matmul(q, k.transpose(2, 3))
```

```

# compute softmax
att = F.softmax(att, dim=-1, dtype=torch.float32)

# convert back to bf16
att = att.to(q.dtype)

# mma att@V
attn_output = torch.matmul(att, v)

```

ThunderKittens 实现：

```

// imports
using namespace kittens;

rt_bf<16, 64> k_reg, v_reg;

// load k from shared memory to register
load(k_reg, k_smem[subtile]);

// compute Q@K.T
zero(att);
mma_ABt(att, q_reg, k_reg, att);

// compute softmax (simplified)
sub_row(att, att, max_vec);
exp(att, att);
div_row(att, att, norm_vec);

// convert to bf16 for mma_AB
copy(att_mma, att);

// load v from shared memory to register
load(v_reg, v_smem[subtile]);
auto &v_reg_col = swap_layout_inplace(v_reg);

// mma att@V onto o_reg
mma_AB(o_reg, att_mma, v_reg_col, o_reg);

```

图2：使用熟悉的类 PyTorch 操作在 TK 中实现注意力机制（以分块形式）。

我们从两个方面强调这些抽象对开发者的价值：

- **探索关键折衷：**通过我们的研究，我们辨识出在实现不同类型并行度时若干基础折衷，例如设置分块布局（warp 级别）、占用率调节（块级别）以及线程块启动顺序（网格级别）。通过我们的消融实验（第3节），我们展示了 TK 简化的接口如何赋予用户驾驭这些折衷的控制权。

- **验证抽象的性能**：我们通过提供一系列 kernel 来验证 TK 抽象的有效性——这些 kernel 在多种 AI 操作上能够匹敌或超越先前的 kernel 性能。我们在 GEMM 和注意力推理上达到了 CuBLAS GEMM 和 FlashAttention-3 注意力的性能，并且在注意力反向传播上超过最强基线 10-40%，在状态空间模型上最多快 8 倍，在线性注意力上最多快 14 倍。这些 kernel 由一个小型学术团队编写，其中包括之前没有 CUDA 经验的本科生。这说明我们的抽象既通用又能够始终达到或超越最新水平。我们希望 TK 和我们总结的洞见能够提高 AI 硬件编程的简易性。

我们的贡献包括：(1) 展示了 TK 中一套精简且有主见的抽象在编写简单高效内核方面出人意料的有效性；(2) 提供了一组高性能 AI 内核。TK 内核已被一些机器学习推理提供商和高频交易公司投入生产使用。我们希望 TK 及相关见解有助于提升 AI 内核编程的易用性。

2 GPU 基础知识

图3：GPU 的软件（及硬件）层次结构示意图。

GPU 的任务被划分为称为 kernel 的小程序。一个 kernel 从高带宽内存 (HBM) 加载数据，对其执行计算，并在结束前将结果写回 HBM。在介绍 ThunderKittens 的抽象之前，我们先回顾 GPU 并行性的背景，包括 warp、块和网络级别的并行。我们沿用了 NVIDIA 的术语，并聚焦于 H100 SXM GPU（尽管原理适用于各厂商和不同代的 GPU）。

2.1 GPU 层次结构

GPU 的软件层次结构与其物理硬件层次高度对应（见图3）。下面我们说明其中若干最重要的组成部分和要点。

1. **Warp**：由32个彼此相邻的线程组成的小组（线程束），这些线程在体积小但速度快的寄存器（register）内存中操作数据。这些指令在各种物理执行单元上运行，不同单元专门用于不同的计算操作（如下），且不同线程可同时占用不同的执行单元：

a. **加载/存储单元**：用于将数据载入寄存器或从寄存器写出数据。最新的 GPU 还引入了用于批量加载和存储的专用硬件加速单元。

b. **通用计算管线**：例如用于最大值、最小值的算术逻辑单元（ALU）、用于乘法和加法的融合乘加单元（FMA），以及用于指数等复杂运算的特殊功能单元（XU）。不同管线的吞吐量各不相同。

c. **矩阵乘法加速硬件**：即张量核心（Tensor Core），提供了 GPU 上绝大部分计算能力。

1. **线程块**：多个 warp 组成线程“块”，它们在一个物理内核（称为流式多处理器，SM）上共同执行一个 kernel。虽然每个 SM 只有四个物理执行单元，但每个 SM 可同时运行多达 64 个软件 warp（即“占用率”最高可达64 warp）。位于同一 SM 的 warp 会竞争硬件资源：寄存器、共享内存、指令发射槽，以及计算管线，但它们共同有助于在每个执行单元中保持大量工作流同时运行。warp 之间可通过屏障同步，在同步期间它们无法发射新工作。

重要的是，同一线程块内的 warp 可通过特殊的片上**共享内存**（SMEM）快速通信（H100 上每个 SM 共享内存 227 KB，带宽 33 TB/s）。为提高带宽，SMEM 被划分为 32 个物理“存储体”（bank），可并行提供内存访问。然而，如果不同线程同时访问同一存储体（即发生**存储体冲突**），它们的访问必须串行化，导致访问延迟增加和实际带宽

下降。Hopper 架构中每线程最多可用 255 个寄存器，超过则会溢出到 L1 缓存。SMEM 也可以动态用作 L1 缓存，以加速访问经常使用的内存（例如溢出的寄存器）。

1. **网格**：由多个线程块组成的网格被启动来运行 kernel。H100 SXM GPU 拥有 132 个物理 SM，可同时运行同数量的线程块。尽管单个 SM 可以同时容纳多个线程块，但大多数 AI kernel 通过在单个线程块内容纳更多 warp（提高占用率）即可实现高性能。

同一 GPU 上的线程块共享全局内存资源：容量大但速度较慢的高带宽显存（HBM，80 GB，带宽 3 TB/s）以及较小但更快的 L2 缓存（50 MB，带宽 12 TB/s）。

调度线程块存在开销。首先，启动一个线程块会产生设置成本——该成本在初次 kernel 启动时至少支付一次，如果一个 kernel 不断启动大量线程块，还会反复产生额外开销。其次，如果网格规模设置不当会导致“尾效应”开销：例如在拥有 132 个 SM 的 H100 上执行包含 133 个线程块的 kernel，需要分两批执行——第一批可充分并行运行，第二批仅占 <1% 的效率。

2.2 成本模型

除了减少总工作量外，降低执行时间的另一个关键方法是同时重叠执行多种工作类型。概括以上组件，我们提供一个简化的 GPU 并行成本模型。我们将整个 kernel 执行时间 C_{Overall} 分解为：

$$C_{\text{Overall}} = \underbrace{\max(C_{\text{HBM}}, C_{\text{L2}}, C_{\text{L1}}, C_{\text{Shared}})}_{\text{Memory}}, \underbrace{\max(C_{\text{Tensor}}, C_{\text{ALU}}, C_{\text{FMA}}, C_{\text{XU}})}_{\text{Compute}} + \underbrace{C_{\text{Setup}} + C_{\text{Sync}}}_{\text{Overhead}}$$

其中，内存成本是延迟和带宽的组合，计算成本是延迟和吞吐的组合。

上述模型表示了内存、计算和张量核心开销完美重叠的理想情况。一个 kernel 的实际性能将介于这些组成部分的最大值和总和之间，具体取决于工作负载性质（例如某些操作天然是顺序执行的）以及实现的效率。不论如何，我们的探索将遵循以下指导原则：(1) 尽量降低各部分的个体开销；(2) 提高它们的整体重叠度。

2.3 GPU 编程框架

我们从诸多相关工作中获得启发，这些工作旨在简化 AI kernel 的开发，例如 NVIDIA 的 CUTLASS/CuTe [29] 和 Triton [39] 等。

CUTLASS 提供了大量嵌套的 CUDA 模板，为高度优化的 AI kernel 提供支持 [8, 9, 37]。从根本上说，TK 和 CUTLASS 都是以 C++ 为依托的库，因此两者能够表达出 **相同的** kernel。我们采取了一种互补的方法，即在抽象上持 **更有主见** 的立场。我们提出的问题是：(1) 少量模板能实现多大的覆盖面？(2) 精简是否会以性能为代价？一个令人期待的结果是提高 AI 研究人员的可及性，因为完全发挥 CUTLASS 能力具有相当挑战性 [9]。我们发现即便是工业界广泛使用的基于 CUTLASS 编写的 kernel（如 FlashAttention-3），也仍存在诸如存储体冲突等本可避免的问题。我们希望提供的抽象能够为用户管理好此类问题。目前多数新出现的 AI 架构使用更高级的编译器工具 [13, 20, 44] 来开发。

Triton、PyTorch [31]、TVM [10]、TensorFlow XLA [1] 等框架从编译器角度切入这一问题。这些框架并非基于 C++ 嵌入式，实现上难以利用未支持的专用硬件指令。在高级框架中，管理异步执行和寄存器使用也很困难。在接下来的章节中，我们探讨一种既保持简单、类似 PyTorch 的使用感，又能实现高性能的途径。更详细的相关工作讨论见附录 A。

3 ThunderKittens

我们提出 ThunderKittens (TK)，这是一个旨在简化高性能 AI 内核开发、同时充分利用现代 GPU 全部能力的框架。本节将 (1) 介绍我们的关键编程抽象，(2) 展示这些抽象如何帮助开发者在不同类型的并行度间进行折衷权衡。第3.1节关注 warp 级别，第3.2节关注线程块级别，第3.3节关注网格级别的并行。

3.1 使用熟悉的数据结构与操作实现线程束级别并行

从本质上说，ThunderKittens 构建在两个基本抽象之上——在各级别上的分块数据结构，以及分块上的操作。TK 提供了多种类型的分块：

- **寄存器分块**：在寄存器（最快的 GPU 内存）中的固定大小 tile，例如 `rt_bf<16, 64>` 表示 16×64 的 BF16 类型 tile，采用列主布局（column-major）。
- **共享内存分块和向量**：模版参数化类型和形状，映射到共享内存。

这些抽象使 TK 能在编译期检查布局和操作，这点很重要，因为 GPU kernel 常常难以调试。例如，执行寄存器内张量核心乘法指令 `mma_AB` 要求 A 分块为行主（row-major）布局、B 分块为列主布局，如果不满足这些条件 TK 可以在编译期报错。

选择内存布局：布局指定逻辑数据元素如何映射到物理线程的拥有关系。不同的执行单元、分块大小和类型，以及硬件加速指令各自偏好不同的布局。不良的布局选择可能导致存储体冲突（参见第2节的 `$C_{\text{Shared}}$`）。我们的目标包括：(a) 默认情况下，我们希望寄存器分块（GPU 中最快的内存）采用满足张量核心（GPU 中最快的计算单元）要求的布局。如图4 左上所示，每种颜色代表数据元素归属于不同线程，张量核心所需的特殊格式往往难以使用和理解（图4 对 naive 布局的讨论有所体现）。(b) 我们希望支持使用硬件加速指令（例如异步矩阵乘法指令和批量拷贝指令），这些指令也要求特定的共享内存布局。

在 TK 中，我们将可能的布局简化为 3 种——内存步长分别为 32、64 和 128 字节——并自动为共享内存分块选择其支持的最大步长布局，以最大程度减少存储体冲突。如第4.2节突出显示的，即使是使用 CUTLASS 和 CuTe 模板编写的经过优化的 FlashAttention-3 kernel 仍遭受存储体冲突，影响性能。而我们的方法能够帮助尽量减少冲突。

图4：共享内存存储体布局示意（以 16×64、16 位元素的分块为例）。左上：朴素的行主（row-major）布局。虽然按行加载效率高，但加载到张量核心所需格式时会产生 8 路的存储体冲突。右上：填充（padded）布局，无存储体冲突但会占用额外内存且硬件支持较差。下方：TK 采用的三种布局中的两种，编译期根据宽度选择（对于某些分块大小，在保持良好硬件支持的前提下仍无法完全避免存储体冲突）。这两种布局分别产生 2 路冲突和 0 冲突。

3.2 使用通用异步模板实现线程块级别并行

ThunderKittens 通过协调线程块内各工作线程组的异步重叠执行，帮助开发者降低开销。尽管 GPU 层次结构可能暗示我们需要大量不同的技术，我们却提出单一简洁的模板，惊人地发现在相当广泛的 AI 工作负载上它都能实现高性能。我们首先定义这个模板——包含四个阶段：加载-计算-存储-结束（简称 LCSF）——它建立在经典的生产者-消费者范式【7, 16】之上。随后我们展示 LCSF 模板如何帮助开发者在占用率和效率之间权衡。

Load 函数：

```

if (warpgroup::warpid() == 0) {
    tma::expect(inputs_arrived,
                block.k, block.v);
    tma::load_async(block.k, globals.k,
                    {batch, head, iter, 0},
                    inputs_arrived);
    tma::load_async(block.v, globals.v,
                    {batch, head, iter, 0},
                    inputs_arrived);
} else {
    arrive(inputs_arrived);
}

```

Compute 函数：

```

warpgroup::mm_ABt(att, scratch.q[state.id], block.k);
warpgroup::mma_async_wait();

// softmax (simplified)
sub_row(att, att, max_vec);
exp(att, att);
div_row(att, att, norm_vec);

copy(att_bf16, att);

warpgroup::mma_AB(state.o, att_bf16, block.v);
warpgroup::mma_async_wait();
arrive(inputs_finished);

```

图5：在 LCSF 模板中的一个注意力 kernel。左侧显示了专注于管理 HBM 与 SRAM（共享内存）数据传输的工作线程功能，右侧显示了在高速内存（寄存器和共享内存）中执行并行计算的工作线程功能。

编程抽象：正如第2节所讨论的，AI kernel 的一般模式是将大张量分块从 HBM 加载到 SRAM，在高速内存中进行计算，然后将该分块结果存回 HBM，并对下一个分块重复此过程。为使用 LCSF 模板，开发者需要编写四个函数：

1. **Load 函数：**指定“加载”工作线程应从 HBM 加载哪些数据到共享内存，以及何时通知“计算”工作线程数据已准备就绪。
2. **Compute 函数：**指定“计算”工作线程应执行的 kernel 指令，使用第3.1节介绍的分块数据结构和操作原语。
3. **Store 函数：**指定由“存储”工作线程需要将哪些数据存回 HBM。

4. **Finish 函数**：在 kernel 结束时执行，工作线程完成最后状态的存储并退出。

TK 提供了一些抽象帮助开发者管理工作线程之间的重叠执行和同步：

- **多级缓冲**：模板在共享内存中维护 N 阶段的流水线缓冲，用于从 HBM 加载和向 HBM 存储数据。加载/存储线程根据计算线程的进度在缓冲中添加/移除数据分块。如果只有单级缓冲，那么在替换输入分块前，加载线程必须等待所有计算线程处理完毕。有了两级缓冲，当计算线程处理当前分块时，下一个分块已可异步加载，从而隐藏 HBM 的加载/存储延迟。更深的缓冲则可减少计算线程间所需的同步，使它们能够并发处理不同的分块。

TK 允许用户设置一个参数来指定阶段数，并为用户管理这些缓冲的创建和使用。如第3.2节所示，我们对 GEMM kernel 在 $N \in \{1, 2, 3, 4\}$ 阶段下进行了性能对比实验。

表1：流水线缓冲阶段数对 GEMM kernel 性能的影响（矩阵规模 $M=N=K=4096$ ）。下表给出了不同阶段数对应的效率（TFLOPS）。

阶段数	TFLOPS
1	260
2	484
3	683
4	760

- **同步屏障**：加载/存储线程需要新的数据写入共享内存缓冲后通知计算线程；而当计算线程处理完分块并将结果写入共享内存后，又需要通知加载/存储线程可以执行存储操作。TK 提供了抽象接口（例如 `arrive` 和基于 `inputs_arrived`、`inputs_finished` 的同步）帮助开发者轻松管理这些同步点。

（注：其余与线程块模板相关的高级选项详见原文。）

3.3 网格级别并行与线程块调度

接下来，在网格级别，我们探讨如何协调线程块的启动顺序。TK 的模板并不强制为用户选择网格结构，但我们提供了两类关键优化的权衡研究：(1) 减少每个线程块的启动和终结开销 ($\$C_{\text{Setup}}\$$)；(2) 提高线程块之间的内存重用以避免慢速的 HBM 访问 ($\$C_{\text{HBM}}\$$)。

表2：持久化线程块对 TK GEMM kernel 的性能影响（TFLOPS）。比较启用（Yes）和不启用（No）持久化启动，在不同矩阵 K 维度下的结果。矩阵规模固定 $M=N=4096$ 。

K	TK-No (TFLOPS)	TK-Yes (TFLOPS)	CuBLAS (TFLOPS)
64	93	108	69
128	161	184	133
256	271	309	242

K	TK-No (TFLOPS)	TK-Yes (TFLOPS)	CuBLAS (TFLOPS)
512	414	450	407
1024	565	600	633

线程块启动开销：我们首先研究“持久化网格”的使用：一次性在所有 132 个 SM 上启动足够的线程块，kernel 内部通过循环让已存在的线程块处理后续工作，而不反复启动新块。我们还探索了另一优化：让线程块在完成当前分块计算的“结束”阶段时，为下一分块预加载输入数据至共享内存缓冲的“输入”阶段。表2 展示了这些优化对我们 GEMM kernel 的性能提升。启用持久化网格后，可显著减少线程块启动的频繁开销。

L2 重用与线程块启动顺序：请回忆线程块需要通过 HBM 进行通信（参见第2节）。当线程块重用数据时，这些数据往往已缓存在片上 L2，从而比从 HBM 获取快得多。然而，由于缓存驱逐机制，重用效果取决于线程块启动的顺序。对于我们的注意力和 GEMM kernel，我们测量了在不同线程块执行顺序下的效率，结果汇总在表3 中。线程块执行顺序显著影响 L2 重用（通过 HBM 带宽反映），而这反过来会左右 kernel 性能。

表3：线程块顺序对缓存重用和性能的影响（提取了注意力前向、GEMM 等场景的 HBM 带宽与 TFLOPS）。

块顺序	HBM 带宽 (GB/s)	TFLOPS
{8, N, M/8}	982	805
{N, M}	3070	392

在如上所示的注意力前向 ($D=128$) 的对比中可以看到，不同块顺序导致的 L2 重用差异对性能影响巨大：一种优化顺序相比天真的行主序执行，使得 $4096 \times 4096 \times 4096$ 的矩阵乘性能几乎未下降（从 767 TFLOPS 降至 735 TFLOPS）；而在 $16384 \times 16384 \times 16384$ 的矩阵乘中，性能则从 797 TFLOPS 骤降到 387 TFLOPS（降低了超过 50%）。可见，线程块调度对于大型问题的性能至关重要。

4 实验

在实验中，我们验证 ThunderKittens 能加速各种 ML 基础算子。我们将 TK kernel 与先前工作中在其他框架（如 Cutlass、CuBLAS、原生 CUDA、Triton）编写的优化 kernel 进行对比。我们选择 AI 中的“工作马”算子（GEMM 和注意力），以及新兴的架构对应的 kernel（如线性注意力和状态空间模型，第4.1节），并与最强的已有实现比较性能。此外，我们使用 NVIDIA H100 80GB SXM GPU、CUDA 12.6，对每个 kernel 进行基准测试（10 次预热+10 次正式计时，C++ 计时），并报告平均的 TFLOPS【注2】。在第4.2节，我们通过分析性能剖析结果来理解 TK 抽象如何实现高性能。附录B 列出了基于 TK 模板编写的内核示例。

【图8+embed_image】**图8：**注意力（因果推理、非因果推理和反向传播）的效率比较。

4.1 TK 实现了简洁且高性能的 AI 内核

【图7+embed_image】**图7：**CuBLAS 与 TK 的 GEMM kernel 性能对比。

本节展示我们在 TK 框架中开发的一系列 kernel。我们在 NVIDIA H100 80GB SXM GPU 上对这些 kernel 进行基准测试（CUDA 12.6），报告平均 TFLOPS 值。

核心算子：多年来，工业界和研究人员在优化 GEMM 和注意力这两大 AI “工作马”算子上投入了巨大精力【8, 15, 30, 37】——它们是 Transformer 架构【41】的两项支柱性操作。尽管投入巨大，我们发现使用 TK 抽象和 LCSF 模板编写的 kernel 完全可以媲美或超越最强基线：

- **线性注意力：**我们优化了两类不同的线性注意力架构，一类使用基于多项式的特征映射【4, 5, 26, 28】，另一类使用可学习的特征映射【45, 46】。在图9 中，我们与当前可获得的最强基线进行比较：即流行的 Flash Linear Attention (FLA) CUDA kernel【44】（使用 Triton 编写）。结果显示，TK 的实现相对于 FLA 的多项式映射线性注意力速度提升了 14 倍，相对于 FLA 的可学习映射线性注意力提升了 6.5 倍。
- **状态空间模型：**长卷积（通过卷积定理用傅里叶变换实现）是流行的状态空间模型架构（如 S4、H3、Hyena【2, 19, 22, 23, 33】等）的核心算子。在图9 中，我们将 TK 与最强的现有基线——Fu 等人【20】提供的 FlashFFTConv CUDA kernel——进行比较。结果显示，在序列长度 4096 时 TK 的性能比该基线高出 4.7 倍，在序列长度 1024 时高出 7.9 倍。相比之下，TK 相对 PyTorch 提供的 FFT 操作也有最多 8.7 倍的提速。

【图9embed_image】**图9：**ThunderKittens 内核在各种算子上均表现出色。我们在一块 80GB 的 NVIDIA H100 GPU 上对这些内核进行了基准测试，并报告了 TFLOPS。

(a) 对于 GEMM，我们的 TK 实现（使用一个 kernel 完成全部计算，详见附录 B.1）在所示矩阵规模上与 CuBLAS 表现相当。

(b) 除了常用算子如 GEMM 和注意力达到峰值性能外，TK 还旨在具备可扩展性，以支持新兴的 AI 工作负载。我们已经发布了一系列针对新兴机器学习算子的 kernel，包括线性注意力【27】、FFT 卷积【11】和状态空间模型【22】。如上所述，TK 在线性注意力和状态空间模型上的表现显著优于已有方案。

4.2 内核实现对比

为了进一步比较 TK 与基线 kernel，我们使用 NVIDIA 的 NSight Compute (NCU) 工具对各 kernel 进行了性能剖析【注3】。在表4 中，我们给出了新兴的长卷积算子以及优化良好的注意力反向传播算子的 NCU 剖析，对比各自最强的基线实现。

- **长卷积：**我们对 FlashFFTConv (FC) 和 TK 的长卷积 kernel 在 B=16, D=1024, N=4096 情况下使用 NCU 进行剖析。结果发现 TK 在重叠不同类型工作线程方面效果更佳（表现为 Issue 槽利用率更高、内存停滞更少），且张量核心利用率也提高了 4.1 倍。这得益于我们的 TK 模板，以及 TK 的 warp 组 (warpgroup) 操作（节省了寄存器，并通过 warpgroup 矩阵乘加指令 WGMMMA 建立了共享内存到寄存器的内存流水线）。
- **注意力反向传播：**我们在 B=16, H=16, N=3072, D=128 下比较 FA-3 和 TK。两者在张量核心利用率上相当，但 TK 的 Issue 槽利用率更高，这表明 TK 的占用率调优可能更好。在 HBM 访存方面，TK 实现了更高的内存吞吐，相应地因等待 HBM 而停滞的周期减少了 10%。对于共享内存，TK 导致的停滞周期比 FA-3 少 85%——我们发现 TK 没有发生存储体冲突，而 NVIDIA 的 NCU 剖析报告 FA-3 中出现了最高 9.6 路的存储体冲突。

这些 kernel 剖析凸显了同时管理各类 GPU 并行性的难度，我们希望 TK 能帮助减少这方面的开发工作。附录 B 提供了 TK 内核示例代码。

5 结论

面对将 AI 架构映射到 GPU 硬件的挑战，我们的问题在于使用少量易用的 GPU 编程抽象能达到何种程度。在 ThunderKittens 中，我们针对 GPU 层次结构的每一层都提供了相应的抽象：在线程束级别提供具有管理布局的分块数据结构，在线程块级别提供异步执行的 LCSF 模板。在网络级别，我们强调了持久化块启动和 L2 重用的选项和权衡。一个自然而然的问题是：使用如此精简的抽象编写 kernel 是否会牺牲性能？我们用 TK 实现了种类广泛的 AI kernel，令人兴奋的是，我们的抽象既通用，又始终能达到或超越最新的性能水平。我们对此充满信心，认为简单易用的 AI 硬件编程方式大有可为。

脚注：

¹ 我们以 NVIDIA 为例进行讨论，但上述并行类型在 AMD、Apple 等其他 GPU 架构中同样适用。

² 文中 Triton kernel 基线源自：<https://github.com/Dao-AILab/flash-attention>

³ NVIDIA Nsight Compute 工具下载链接：<https://developer.nvidia.com/nsight-compute>
